面试准备包 • 基于 LoRA 微调后 Qwen3 的 AI 日程助手

一、二部分念到能自然说出来；三部分是两块硬骨头的机制解释；四部分只在卡壳时用。

一、项目介绍（口述稿，2 分钟）

这是一个**本地优先的 AI 日程助手**，核心是基于 **LoRA 微调的 Qwen3 小模型**构建的 「自然语言 → 结构化工具调用」转换层。

产品分两层。**指令级**：一句话加日程，由 0.6B 模型抽取成结构化命令并落库。 **规划级**：贴一段杂乱待办，由 4B 模型先抽取任务与约束，再交给求解器排出一周计划。 分成两个模型是因为**延迟要求不同**：交互要几百毫秒，规划可以容忍几秒。

技术上有三个取舍。第一，**模型只做语义归一化，不做计算**：契约只允许相对时间， 绝对日期解析下移到确定性代码，从根上消除”模型算错日期”这类不可测缺陷。 第二，**规划交给确定性求解器**：CP-SAT 求解只要 20–30ms，排不出时还能定位到具体冲突约束， 既是准确率问题也是可解释性问题。第三，**以单一 JSON Schema 作为唯一事实源**， 同时派生解码文法、训练标签、执行校验与 MCP 工具签名，CI 断言防漂移。

实测在无独显的核显设备上，端到端中位 **641ms**，结构合规 **4372/4372**。

二、模拟 Q&A

2.1 基础部分

Q：这个项目解决什么问题？ A：把自然语言可靠地转成工具调用。难点不在语义理解，而在**输出必须结构合法、语义正确**。 做法是**分离不确定性**：语言理解交给模型，时间解析与排期交给确定性代码。

Q：为什么不用云端大模型？ A：**隐私**（日程是高敏感个人数据）、**成本**（高频轻量调用不经济）、**可用性**（离线可用）。 代价是模型能力受限，所以要用约束解码、规则层等手段补齐可靠性。

Q：为什么用 0.6B 和 4B 两个模型？ A：按延迟分档。交互要几百毫秒，0.6B 在核显上 111 tok/s 够用；规划一次要处理整段素材、 抽取质量比速度重要，用 4B（22 tok/s）换准确率。**一个模型通吃会两头不讨好。**

Q：为什么选 LoRA 微调，而不是直接用更大的基座？ A：实测过基座在当前契约下的表现：1.7B/4B 零样本准确率仅 14%/19%——**换更大的模型解决不了 “不会用新契约”**。LoRA 只训低秩旁路，显存成本低一两个数量级，且能让模型学到训练分布里的 **联合约束**（多条待办、时长与截止的对应关系），这是 few-shot 补不上的。

Q：什么是约束解码？ A：把 Schema 编译成**文法**，推理时每个 token 只能在文法允许的集合里采样，输出在结构上不可能非法。但它**只保证格式，不保证语义**。

Q：CP-SAT 承担什么职责？ A：解带约束的排期：输入任务（时长/截止/优先级）与可用性限制，输出满足全部硬约束、 软约束代价最小的周计划。确定性求解器，结果可复现、可解释。

2.2 技术深入部分

Q：约束解码具体是怎么做到的？ A：模型是自回归逐 token 生成的，默认词表里任何 token 都可能被选中，所以会出 引号不闭合、枚举越界、前后多一句解释这类错。约束解码在**采样那一步干预**： 用文法状态机判断当前位置允许哪些 token，把不允许的 **logit 置为 -∞**——softmax 后概率恰好为 0，不是”很低”而是”不可能”。本质是一个下推自动机在跟踪语法走到哪了。

Q：它的代价是什么？ A：三个。**首 token 延迟**（语法要预编译）；**表达受限**（Schema 没定义的生成不出来， 所以契约设计本身就是产品设计）；**token 边界问题**（tokenizer 词与语法边界不对齐， 状态机要处理”半个 token”）。另外它保证不了语义——“开会”该归 meeting 还是 class， 模型仍可能判错，所以执行层留了 clarify（追问）和 reject（拒答）两个出口。

Q：4B 为什么一度反而更差？ A：Qwen3 默认开 thinking，4B 把 token 预算全花在推理链上——实测 298 个 reasoning chunk、 **正文零输出**。--reasoning-budget 0 **无效**，它只把文本挪到 content，推理没停，还给了错年份。 真正生效的是 --reasoning off，关闭后输出**逐字一致**，1900ms → 641ms。

Q：单一事实源怎么防止契约漂移？ A：四处消费路径不同但来源唯一：约束解码由 Schema 生成文法；训练集生成器据其决定标签字段； 执行层用 Pydantic 镜像校验；MCP 服务端用 Literal[*SCHEMA_ENUM] 直接取枚举。 CI 两条断言：Pydantic 字段集合 == Schema；MCP 工具枚举 =

= Schema 枚举。 这类漂移最坏的地方是**不报错**——训练时一个字段、调用时另一个字段，模型输出看着正常，校验全挂。

Q：反向构造怎么保证标签正确？ A：**先有标签、再渲染文本**，标签从真值直接生成，构造上不会错。但渲染可能漏说 （文本没说时长、标签却写 120 分钟），模型就会学臆测。所以另写校验器逐条断言“标签每个字段都能由文本推断”，且**按分句**检查——全文检查会被其他事项的措辞掩盖。 更关键的是给校验器写自测：注入五类错误标签，断言全部捕获、干净数据零误报。 **一个永远通过的校验器等于没有校验器**——初版踩过：把 priority 改成 high，但样本本已是 high。

Q：规则层和模型怎么分工？会不会改坏模型对的输出？ A：判据是”**规则可做且更可靠的，就不交给模型**”。模型抽 keep_free（禁排时段）仅 30.6%， 而”周三上午别排东西”这类句式正则几乎不出错，接管后到 94.4%。踩过两次坑： 把约束里的”周三”误作事项截止日 → 改为**先抽约束、记下字符区间、抹除后再匹配日期**； 标题与文本不完全匹配时退化全文查找 → 改为**定位失败即不修改**。

Q：现在 39% 怎么上到 80%？ A：短期靠微调，数据集与训练套件已就绪（4000 条反向构造、标签 100% 校验通过），只缺算力。 长期靠**联合分布**：基座逐 token 贪心，没有”整条样本自洽”的约束，导致单字段对、整体不齐。

三、技术扫盲（两块硬骨头）

3.1 约束解码：为什么”格式不可能非法”

先看清楚问题。 模型不是一次性吐出整段 JSON，而是**自回归**逐 token 生成： 每步在整个词表（十几万 token）上按概率挑一个，追加，再算下一个。默认情况下**谁都能被选中**， 所以自由生成会出这些错：

错误	表现
结构断裂	{"module": "meetin 之后直接结束，引号没闭合
枚举越界	"module": "开会" —— 词表里有，但契约只允许 meeting
多余解释	JSON 前后带一句”好的，我帮你安排：…”

根因：格式约束只存在于训练分布里，不存在于生成机制里。 模型只是”倾向于”写对， 没有任何机制**保证**它。而工具调用是 0 容忍的——解析失败就是功能失败。

约束解码把这件事变成数学保证。 它不做事后修补，而是在**采样那一步**干预：

每步：① 状态机给出"当前位置允许哪些 token"
② 不在集合里的，logit 置为 -∞
③ 剩下的做 softmax，再采样

-∞ 的指数是 0，所以非法 token 的概率**不是”很低”，而是恰好为零**。 状态机本质是一个**下推自动机**，在跟踪语法走到哪了：读到 {"module": " 只许枚举里的值， 读到 duration_min": 后只许数字。

> 面试口径：约束解码是在**解码器的采样层**做词表剪枝，用文法状态机屏蔽非法 token 的 logit， 使输出在语法上不可能违规。注意是采样层，不是后处理。

GBNF 是 llama.cpp 的文法描述语言（写法接近正则 + BNF）：

```
root   ::= "{" "\"module\":" module "," "\"title\":" string "}"
module ::= "\"class\"" | "\"exam\"" | "\"meeting\"" | "\"alarm\""
```

但项目里不用手写——llama.cpp 支持**从 JSON Schema 直接编译**（请求带 response_format: json_schema）。**所以只维护一份 Schema，拿到整套语法约束。**

三个代价（面试爱追问）：①**首 token 延迟**，语法要预编译；②**表达受限**，Schema 没定义的 字段根本生成不出来，所以契约设计本身就是产品设计；③ **token 边界问题**，tokenizer 的词 可能与语法边界不对齐，状态机要处理”半个 token”。

但它管不了语义。 能保证 module 落在枚举里，保证不了”开会”该归 meeting 还是 class。 所以执行层再留两个出口：clarify（信息不足反问）与 reject（越界拒答）。 一句话：约束解码解决”**能不能解析**”，这两个出口解决”**该不该执行**”。

3.2 单一事实源：防的是”静默不一致”

同一份结构有四个消费方：

消费方	用途	怎么派生
约束解码	生成文法约束模型输出	把 Schema 交给 llama.cpp 编译
LoRA 训练	决定训练标签的字段与枚举	生成器读 Schema 构造标签
执行校验	Pydantic 校验入参	**镜像类**，字段一一对应
MCP 工具	inputSchema 给客户端看	Literal[*SCHEMA_ENUM] 直接取枚举

各处手写的后果不是”某天突然坏”，而是**慢慢不一致直到某次上线炸掉**：

> 训练时模型学 remind_before，推理时校验要 remind_before_min —— 模型输出完全正确，但校验全挂。而且**不报语法错**，最难查的一类 bug。

这叫**契约漂移**。解法：只有 schema/schedule_command.v2.json 是人写的，其余三处派生。 Pydantic 那处为什么”镜像”而不直接读 Schema？因为运行时要类型化对象与逐字段校验， Schema 只是 JSON 描述——镜像类无法自动生成，所以**用 CI 断言锁死**： ①Pydantic 字段集合 == Schema；②MCP 工具枚举 == Schema 枚举。

机器能验的，就不靠人记。

3.3 MCP 到底管什么（最容易混）

概念	管什么	不管什么
Function Calling	模型调用函数的能力	不管工具怎么被发现
提示词工程	怎么说清楚让模型照做	完全不管格式保证
MCP	**工具如何被描述、发现、调用**	不管中文怎么变 JSON

MCP 是”工具层的标准化”，不是”翻译层”。 价值在于任意 MCP 客户端零适配接入 （客户端自己 tools/list 拿签名，不用读文档、不用写胶水代码）。

我们的亮点是：这个 inputSchema **不是手写的，是与解码文法、训练标签同源派生的，** 所以”模型能生成的 JSON”和”客户端看到的工具签名”永远一致。

四、答不上来时的应对话术

1. **完全没听过 →** “这块我没有深入了解，不敢妄断。我理解它面向 X 类问题，但机制不确定。如果工作中要用，我会先查官方文档与最小可运行示例，再回来确认。”
2. **记得结论忘了推导 →** “结论我记得是 X，但推导需要重走一遍。我可以把测量方法讲一下——你更关心结论还是方法？”
3. **被追问到盲区 →** “这点我确实没考虑到。初步判断影响可能在 A 与 B 两方面，但需要验证。您能指一下是哪个方向吗？”（**把追问变成学习机会**）
4. **数据记不准 →** “具体数值我不保证，量级是 600 多毫秒，准确值在仓库 benchmark 里。 我更确定的是**相对关系**：关掉 thinking 后快了约 3.6 倍，输出逐字一致。”
5. **被指出方案有问题 →** “您说得对，这确实是漏洞。当时我在 X 与 Y 之间选了 X，原因是……。若按您的方案改，我担心引入 Z。您认为 Z 可以接受吗？”（**承认取舍，不硬辩**）
6. **通用兜底 →** “整体我还不确定，但有一点可以确定：_____。其余我想确认后再答。”

五、面试前 5 分钟速览

- 核心叙事：**模型不算日期 / 规划交给 CP-SAT / 单一事实源防漂移 / 反向构造 + LoRA**
- 三个数字：**641ms、4372/4372、4%→39%**
- 三个诚实点：抽取准确率还不够、未做多轮对话、微调因缺算力未跑
- 收尾：**“这个项目我最想展示的不是功能，而是如何把不确定的模型约束进确定的工程边界内。”**